

Object Oriented Programming (OOPs)

Classes and Objects

Java is considered **mostly object-oriented**, but not purely object-oriented because it supports primitive data types (`int` , `char` , `boolean` , etc.) that are not objects.

In object oriented programming language , basically classes and objects are used. Classes are like the blue prints that have properties (also called states) and behaviors while the class is a blueprint. An object is the actual entity created from that blueprint when we create a class then its object is generally created in the main function of the class and that is called that object is instantiated.

Local reference variables are stored in the stack, but instance reference variables are stored inside the object itself on the heap.

OOPs is generally used because it helps in following things:

- Code reusability
- Code maintainability and Scalability
- Code is easier to manage
- and it is closer to the real world entities

example:

```
Car c = new Car();
```

new

↓

Memory allocated on Heap

↓

Constructor executes

↓

Reference stored in Stack

****Characteristics of OOP****

The diagram below demonstrates the Java OOPs Concepts

Object-Oriented Programming

Class

A blueprint or template for creating objects.

Object

An instance of a class.

Encapsulation

Protects data from direct access

Abstraction

Hides unnecessary details

Polymorphism

Allows the same method to behave differently

Inheritance

Reuses code from existing classes

There are derived as well as built in classes too

String is a built in class , similarly date and some other classes

When we instantiate an object in the class then the variable assigned to it in the main function is its reference while the assigned value is where the object is assigned to it and hence object is created.

- A class is a user-defined data type.
- It contains variables (fields) and methods.
- Multiple objects can be created from a single class.

Whereas an object is used to access the state and behaviors that are defined by the user in the class and assigned a unique property to it as there can be multiple objects of a single class.

- ****State****: Represents the current data or attributes of an object.
- ****Behavior****: Represents the actions that an object can perform.
- ****Identity****: Every object has a unique identity in memory that distinguishes it from other objects, even if they contain the same data.

Instance variables: An **instance variable** is a variable declared inside a class but outside any method, constructor, or block.

- Memory for instance members is allocated when an object is created, while static members are allocated when the class is loaded into memory.
- Objects mirror real-world items such as customer, product or circle. Objects are created in the heap memory, while reference variables are stored in the stack (when they are local variables). The reference variable holds the memory address of the object in the heap.

****Note:**** Every class has at least one constructor. If none is defined Java provides a default no-argument constructor that calls the parent constructor.

Advantage of OOP over Procedure-Oriented Programming Language

Object-oriented programming (OOP) offers several key advantages over procedural programming:

- ****Code reusability:**** Classes and objects allow reuse of existing code, reducing duplication and improving efficiency.
- ****Better structure and maintainability:**** Programs are organized into logical units, making code easier to understand, debug, and maintain.
- ****Supports DRY principle:**** Common functionality is written once and reused, leading to cleaner and more maintainable code.
- ****Faster development:**** Modular and reusable components help in quicker and scalable application development.

Limitations of OOP

While OOP provides many benefits, it also has some limitations:

- ****Steeper Learning Curve:**** Concepts such as classes, objects, inheritance, polymorphism, abstraction, and encapsulation can be challenging for beginners.
- ****Additional Overhead for Small Programs:**** OOP may introduce extra classes and structure that are unnecessary for simple applications.
- ****Increased Design Complexity:**** Designing a proper class hierarchy and object relationships requires careful planning.
- ****Higher Memory Consumption:**** Creating and managing a large number of objects can require more memory compared to procedural approaches.

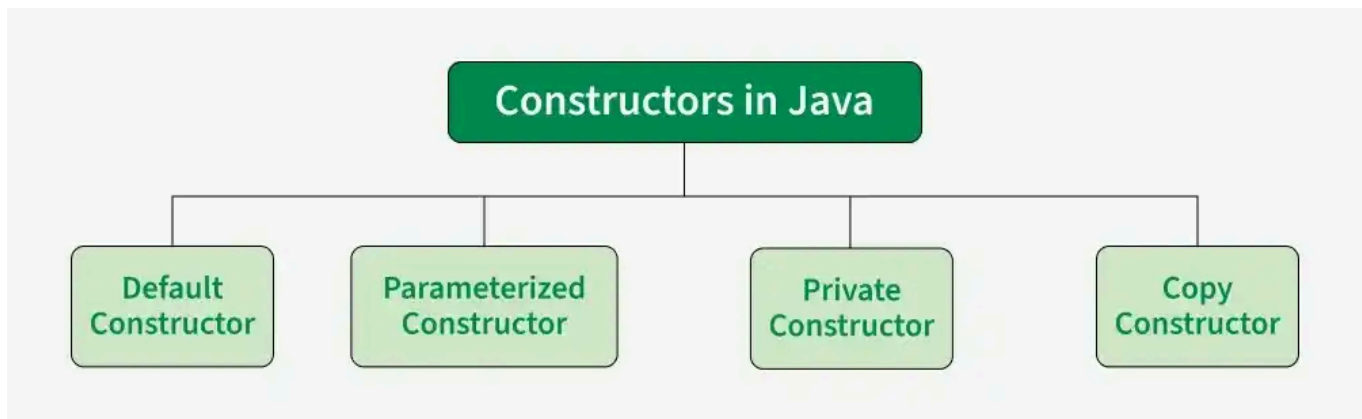
Constructor

Whenever an object is created, the constructor is a member of a class that gets automatically invoked. It is used to initialize the object's data member and set up the initial state of an object.

- Constructor name must be the same as the class name.
- It does not have any return type, not even void.
- It is automatically called when an object is created using the new keyword.
- Constructors help initialize object properties during object creation.

Java provides a default constructor without any parameter although yes if we have defined any constructor then it will not provide the default constructor

Types of Constructor



- ****Default Constructor:**** A constructor that does not accept any parameters and initializes an object with default values.
- ****Parameterized Constructor:**** A constructor that accepts parameters to initialize an object with specific values.
- ****Copy Constructor:**** A user-defined constructor that creates a new object by copying the data of another object of the same class.
Java does not have a built-in copy constructor. A copy constructor is a user-defined constructor that copies the fields of another object.
- ****Private Constructor:**** A constructor declared with the private access modifier that prevents object creation from outside the class.

```
this(...)
```

and

```
super(...)
```

cannot both appear in the same constructor because **both must be the first statement**.

1. Abstraction

It is basically the process of hiding the implementation details and just focuses on showing the important features of an object.

It focuses on what the object does rather than how it does it

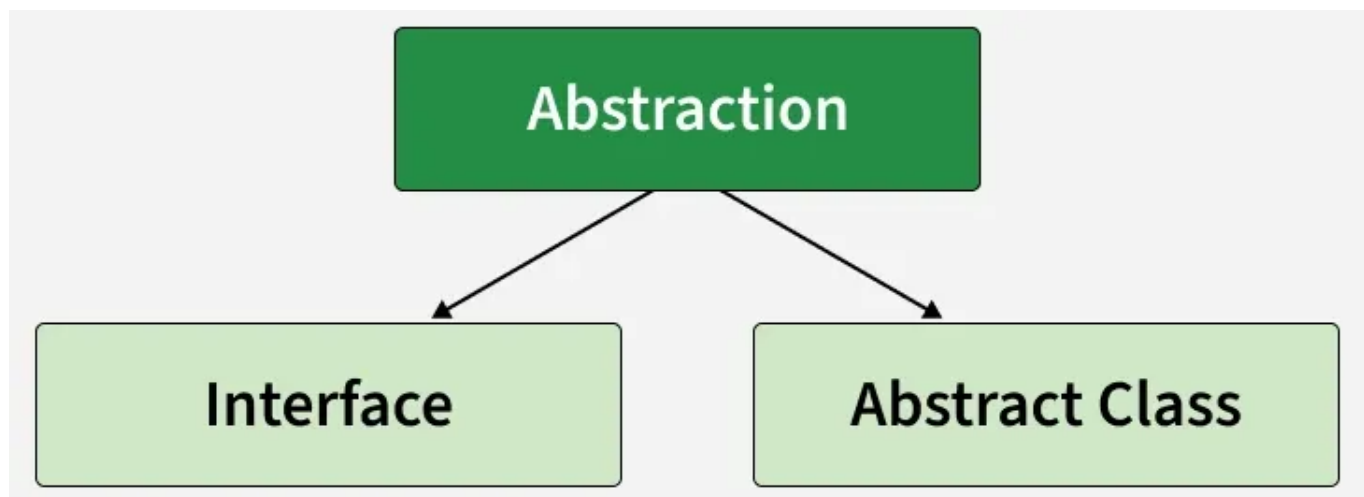
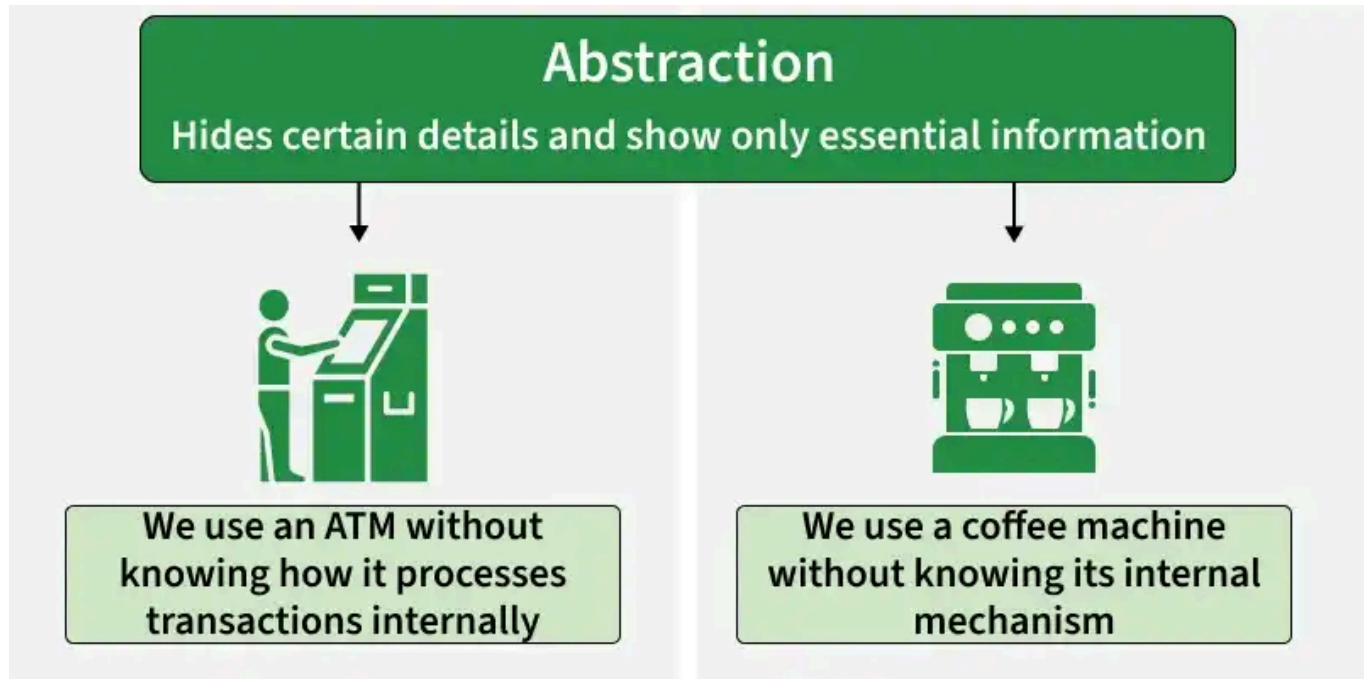
Things it helps with are:

- It helps in code flexibility as in the implementation of the code can be changed while disrupting how the user uses it
- It ensures code flexibility by loose coupling

- It also hides complexity by just showing what the object does rather than how

It achieves this by abstract classes and interfaces, while interfaces ensures 100% of abstraction abstract classes achieve partial abstraction.

Interfaces primarily provide abstraction and define contracts. Since Java 8, interfaces can also contain default, static, and private methods.



Abstract Classes

Abstract class is a class that cannot be instantiated which means its object cannot be created and it has to be implemented by its child classes.

A concrete subclass must implement all inherited abstract methods. An abstract subclass may leave them unimplemented.

- It helps in achieving partial abstraction as it also contains concrete methods and properties, final and static methods, constructors ,instance variable along with abstract methods.

Syntax:

```
abstract class ClassName {  
    abstract void methodName(); // Abstract method  
    void concreteMethod() { // Concrete method  
        System.out.println("Method implementation");  
    }  
}
```

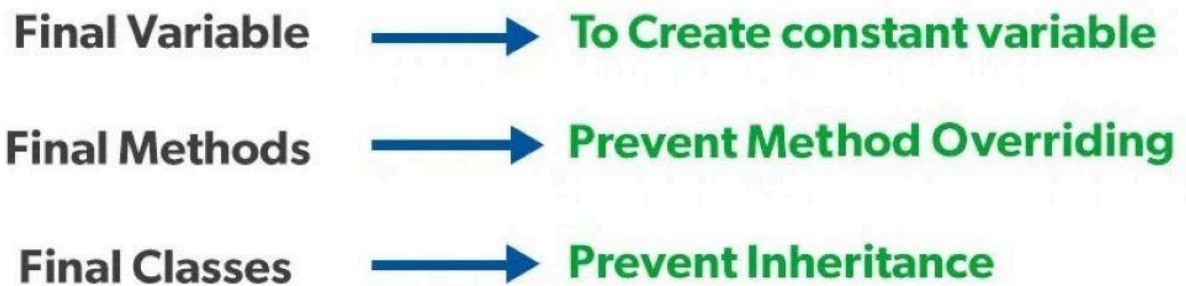
Final keyword

In Java we use the final keyword when want to restrict the changes in the class ,method or variable.

It can be applied to variables, methods, and classes to prevent modification, overriding, or inheritance. This helps in creating constant values, stable methods, and immutable classes. The final keywords help in making the code more secure

- Final variable cannot be changed once assigned
- Final method cannot be overridden
- Final class cannot be inherited

The following are different contexts where the final is used:



Characteristics of final keyword in Java

- ****Final variables**** hold a value that cannot be reassigned after initialization.
- ****Final methods**** cannot be overridden by subclasses.
- ****Final classes**** cannot be extended.
- ****Initialization rules**** require that a final variable must be assigned exactly once either at declaration or inside constructors or initializer blocks.

- ****Reference final variables**** cannot change which object they point to though the internal state of the object can change.
- ****Static final variables**** represent constants shared across all objects.
- ****Blank final variables**** are declared without initialization and must be assigned once before use.
- ****Local final variables**** inside methods must be initialized within their block.

Ways we use the final keyword in Java

The final keyword is used in exactly 3 main contexts:

1. Final Variable

A variable declared with final becomes constant after one assignment.

2. Final class

A class declared as final cannot be extended. A final class cannot be inherited, meaning no other class can extend it. This is commonly used for security or when the implementation should remain unchanged, such as in core classes like String.

- Prevents inheritance, ensuring the class behavior cannot be modified
- Commonly used for immutable and secure classes

3. Final Method

When a method is declared with final keyword, it is called a final method in Java. A final method, on the other hand, can be inherited but cannot be overridden by subclasses, ensuring that the original implementation remains intact.

- Cannot be overridden in subclasses
- Helps maintain consistent behavior across all derived classes

Advantages of final Keyword

- Supports immutability by preventing reassignment
- Helps compiler and JVM optimize code in some scenarios
- Makes behavior predictable since values or methods stay unchanged
- Prevents accidental or unauthorized modification of critical logic
- Preserves API contracts by avoiding unwanted overriding

INTERFACE

In Java ,an interface provides a blueprint for a set of methods a class must implement without showcasing the implementation details.

It helps achieve abstraction by focusing on what a class should do rather than how it does it. Interfaces also support multiple inheritance in Java.

Interfaces support loose coupling.

Frameworks like Spring Boot heavily depend on interfaces because they allow implementations to be swapped without changing client code.

- A class must implement all abstract methods of an interface.
- All variables in an interface are public, static, and final by default.
- Interfaces can have default, static, and private methods (Java 8+ and 9+).

****Notes:****

- Private methods can only be called inside default or static methods.
- Static methods are accessed using the interface name, not via objects.
- To implement an interface, use the implements keyword.

Interface is generally used when the user wants to implement abstraction or multiple inheritance.

Advantages of Abstraction

- Abstraction makes complex systems easier to understand by hiding the implementation details.
- Abstraction keeps different part of the system separated.
- Abstraction maintains code more efficiently.
- Abstraction increases the security by only showing the necessary details to the user.

Disadvantages of Abstraction

- It can add unnecessary complexity if overused.
- May reduce flexibility in implementation.
- Makes debugging and understanding the system harder for unfamiliar users.
- Overuse of abstraction may make code harder to follow, though performance impact is usually negligible.

Common Mistakes to Avoid

The common mistakes that can occur and we should avoid when working with Abstraction in Java are listed below:

- ****Not Implementing Abstract Methods:**** Always make sure that the abstract methods are implemented in the concrete subclass.
- ****Overusing Abstraction:**** Avoid making everything abstract when it's not required. Use abstraction only when it enhances the design.
- ****Inconsistent Method Signatures in Subclasses:**** When you override abstract methods, please make sure the method signature matches exactly, any mistake can cause errors.

Encapsulation

It is that OOPs principle that binds the properties and methods together in a single entity specifically a class and ensures data security, the data here can be generally accessed through getter and setters, if the members are declared private.

It restricts the direct access to the data through hiding the implementation details.

Encapsulation is wrapping data and methods into a single unit while restricting direct access using access modifiers.

- Achieved using access modifiers like private, protected, and public.
- Improves data security by allowing validation through getters and setters.
- Enhances code maintainability by isolating changes within the class.

```
class Programmer {
```

```
    private String name;

    // Getter method used to get the data
    public String getName() { return name; }

    // Setter method is used to set or modify the data
    public void setName(String name) {

        this.name = name;
    }
}
```

```
}
```

```
public class Geeks {
```

```
    public static void main(String[] args){

        Programmer p = new Programmer();
        p.setName("Geek");
    }
}
```

```
        System.out.println("Name=> " + p.getName());  
    }  
}
```

****Advantages of Encapsulation****

The advantages of encapsulation are listed below:

- ****Data Hiding:**** Encapsulation restricts direct access to class variables, protecting sensitive data from unauthorized access.
- ****Improved Maintainability:**** Changes to internal implementation can be made without affecting external code that uses the class.
- ****Enhanced Security:**** Encapsulation allows validation and control over data, preventing invalid or harmful values from being set.
- ****Code Reusability:**** Encapsulated classes can be reused in different programs without exposing internal logic.
- ****Better Modularity:**** Encapsulation promotes organized, modular code by keeping data and methods together within a class.

****Disadvantages of Encapsulation****

The disadvantages of encapsulation are listed below:

- ****Increased Code Complexity:**** Writing getter and setter methods for every variable can make the code longer and slightly more complex.
- ****Performance Overhead:**** Accessing data through methods instead of directly can introduce a minor performance cost, especially in performance-critical applications.
- ****Less Flexibility in Some Cases:**** Over-restricting access to class members may limit the ability of other classes to extend or use the class efficiently.

Data Hiding vs Encapsulation

Aspect	Data Hiding	Encapsulation
Definition	Restricting direct access to data to protect it from unauthorized use	Wrapping data and methods into a single unit (class)
Focus	Security of data	Structure and design
Purpose	Prevents misuse and accidental modification of data	Improves modularity and maintainability
How it is achieved	Using access modifiers (private, protected)	Using classes, methods, and access modifiers

Aspect	Data Hiding	Encapsulation
Involves methods	No (mainly focuses on data)	Yes (includes both data and methods)
Scope	Narrow concept	Broader concept
Dependency	Independent concept	Encapsulation may use data hiding for better security but it is not mandatory.
Example	private int balance;	class Account { private int balance; public getBalance(); }

Inheritance

It is the property of the OOPs in which one class inherits the properties and methods of another class without rewriting the code promoting code reusability and maintainability

- A subclass can reuse the fields and methods of the parent class without rewriting the code
- A subclass can add its own fields and methods or modify existing ones to extend functionality.

```
class Parent {
// fields and methods
}
class Child extends Parent {
// additional fields and methods
}
```

****Note:**** In Java, inheritance is implemented using the extends keyword.

Inheritance → IS-A

Aggregation → HAS-A

Composition → PART-OF

Types of Inheritance in Java

It defines the different ways a class can inherit properties and behavior from one or more classes.

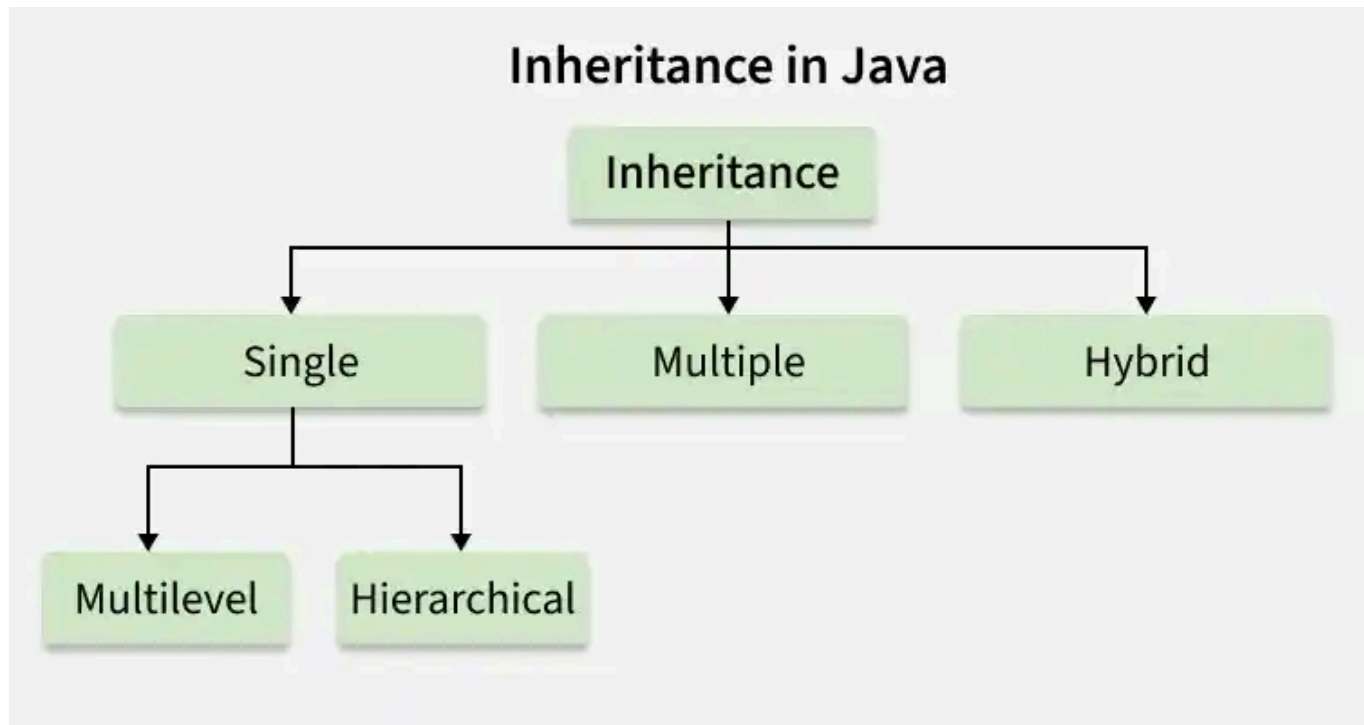
Inheritance

↓

Represents an IS-A relationship.

Example

Dog IS-A Animal



Java supports the following types of inheritance:

- ****Single Inheritance:**** One subclass inherits from one superclass.
- ****Multilevel Inheritance:**** A class is derived from another derived class, forming a chain.
- ****Hierarchical Inheritance:**** Multiple subclasses inherit from a single superclass.
- ****Multiple Inheritance (through Interface):**** A class inherits from multiple interfaces since Java does not support multiple inheritance using classes.
- ****Hybrid Inheritance (through Interface):**** A combination of two or more types of inheritance, achievable using interfaces.

Super Keyword

The super keyword in Java is used to refer to the immediate parent class object in an inheritance hierarchy. It allows a subclass to explicitly access parent class members when they are hidden or overridden. This keyword helps maintain clarity and control while working with inheritance.

- Used to call parent class constructors using `super()`.
- Helps access parent class methods and variables when overridden or hidden.
- Ensures proper inheritance behavior and code reusability

Advantages of Using Java Super Keyword

The advantages of super keyword are listed below:

- With the help of super keyword, subclasses can inherit the functionality from their parent classes.
- Subclasses can override methods and can access fields and methods from their parent class with the help of super keyword, because of this the code becomes more flexible.
- With the help of super keyword we can easily access the methods and fields from the parent class without recreating it in the subclass.
- With the help of super keyword we can achieve abstraction and encapsulation. Subclass can focus on their specified tasks and the parent class take care of the general functionality.

Important Points

- super() must be the first statement in a subclass constructor.
- If no superclass constructor is explicitly called, Java automatically inserts a call to the no-argument constructor.
- If the superclass does not have a no-argument constructor, compilation fails.
- Constructor calls form a chain, ending with the Object class constructor ([constructor chaining](#)).

What Can Be Done in a Subclass?

In sub-classes we can inherit members as is, replace them, hide them or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- We can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- We can write a new instance method in the subclass that has the same signature as the one in the superclass, thus [overriding](#) it (as in the example above, toString() method is overridden).
- We can write a new static method in the subclass that has the same signature as the one in the superclass, thus hiding it.
- We can declare new methods in the subclass that are not in the superclass.
- We can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword [super](#).

Advantages of Inheritance in Java

- ****Code Reusability:**** Inheritance allows for code reuse and reduces the amount of code that needs to be written. The subclass can reuse the properties and methods of the

superclass, reducing duplication of code.

- ****Abstraction:**** Inheritance allows for the creation of abstract classes that define a common interface for a group of related classes. This promotes abstraction and encapsulation, making the code easier to maintain and extend.
- ****Class Hierarchy:**** Inheritance allows for the creation of a class hierarchy, which can be used to model real-world objects and their relationships.
- ****Polymorphism:**** Inheritance allows for polymorphism, which is the ability of an object to take on multiple forms. Subclasses can override the methods of the superclass, which allows them to change their behavior in different ways.

Limitations of Inheritance in Java

- ****Complexity**** -> Deep inheritance hierarchies can make code hard to understand .
Correction -> keep hierarchy simple or use composition instead
- ****Tight Coupling**** -> Changes in superclass can affect subclasses . Correction -> use loose coupling (interfaces or composition)

Polymorphism

This concept of OOPs means an object or method can take multiple forms depending upon the context.

- Same method, different behavior depending on the object
- Achieved through method overloading and method overriding

Reference Type

↓

Determines what members are accessible.

Actual Object Type

↓

Determines which overridden method executes.

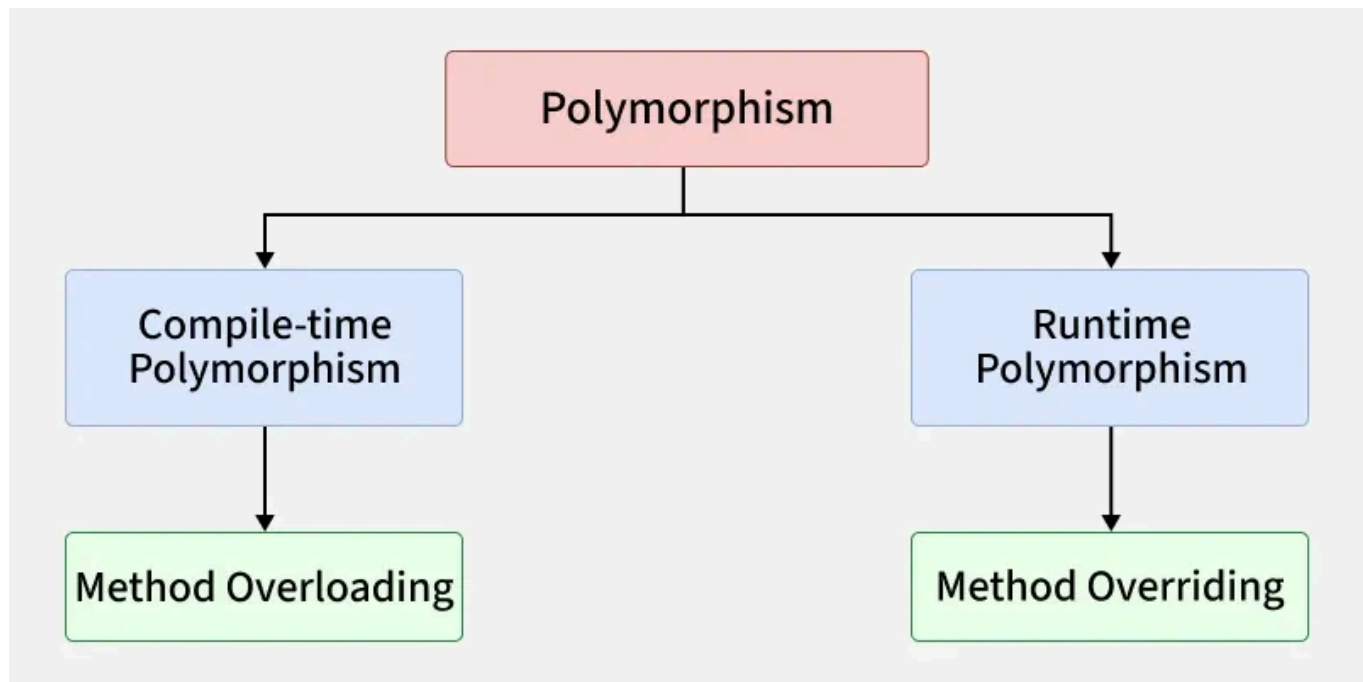
Example:

```
Animal a = new Dog();
```

- Accessible members → `Animal`
- Executed overridden method → `Dog`

Types of Polymorphism

Polymorphism in Java is mainly of 2 types as mentioned below:



1. Compile-Time Polymorphism

[Compile-time polymorphism](#) also known as static polymorphism, occurs when the method to be executed is determined during compilation. It is primarily achieved through method overloading.

- Method selection occurs at compile time.
- Improves readability and flexibility.

****Method Overloading****

[Method overloading](#) allows multiple methods with the same name but different parameter lists within the same class. The compiler chooses the appropriate method based on the arguments passed.

Method overloading rules include:

- Different parameter list is required.
- Return type alone cannot distinguish overloaded methods.
- Access modifiers don't matter for overloading.

****Syntax:****

```
class Demo {  
    void show(int a) {  
    }  
    void show(int a, int b) {
```

```
}  
}
```

2. Runtime Polymorphism

Runtime polymorphism also known as [dynamic polymorphism](#), occurs when the method call is resolved during program execution. It is achieved through method overriding.

Runtime polymorphism is implemented using

Dynamic Method Dispatch.

- Method selection occurs at runtime.
- Depends on the actual object type.
- Supports late binding (dynamic binding).
- Enables one interface to represent multiple implementations.
- Widely used in frameworks and API design for generic programming

****Method Overriding****

[Method overriding](#) occurs when a subclass provides its own implementation of a method already defined in its superclass. The overridden method is selected based on the actual object created.

Method overriding rules Include:

- Cannot reduce visibility.
- Return type can be covariant,
class Animal {}

```
class Dog extends Animal {}
```

```
class Parent {  
    Animal get() { ... }  
}
```

```
class Child extends Parent {  
    Dog get() { ... }  
}
```

Rule: An overriding method may return a subtype of the original return type.

- Cannot override `final`, `static`, or `private` methods.
- Can throw the same or narrower checked exceptions.

private methods

↓

Cannot be overridden

They are not inherited.

These come up often.

****Syntax:****

```
class Parent {  
    void print() {  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void print() {  
    }  
}
```

****Note:**** At runtime, the method that gets executed depends on the actual object type, not the reference type.

Advantages of Polymorphism

- ****Code Reusability:**** The same method or class works with different object types.
- ****Flexibility:**** Different objects can be treated as a common superclass type.
- ****Abstraction:**** Enables working with general types (abstract classes or interfaces) instead of concrete classes.
- ****Dynamic Behavior:**** Method calls are resolved at runtime based on the actual object type.

Disadvantages of Polymorphism

- It can make more difficult to understand the behavior of an object.
- This may cause performance issues, as polymorphic behavior may require additional computations at runtime.

Record in java(data holders)

In Java, a record is a special type of class declaration aimed at reducing the boilerplate code. Java records were introduced with the intention to be used as a fast way to create data carrier classes, i.e. the classes whose objective is to simply contain data and carry it between

modules, also known as POJOs (Plain Old Java Objects) and DTOs (Data Transfer Objects). Record was introduced in Java SE 14 as a preview feature, which is a feature whose design, implementation, and specification are complete but it is not a permanent addition to the language, which means that the feature may or may not exist in the future versions of the language. Java SE 15 extends the preview feature with additional capabilities such as local record classes.

They automatically generate methods such as equals(), hashCode(), and toString(), as well as accessor methods for the fields.

Records cannot extend another class.

```
record Student(String name, int age) {}
```

Records are **implicitly final**, so they **cannot be extended**.

They implicitly extend java.lang.Record.

```
{{{
```

```
equals()
```



If two objects are equal



Their hashCode() must be equal.

=====

compares references

```
equals()
```

compares contents

```
}}}
```

- **Access Modifiers:**

Access modifiers are basically the one that decides whether the data or method or class would be accessible outside the class or within the class.

Modifier	Same Class	Same Package	Subclass	Other Package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗ (unless through inheritance)
public	✓	✓	✓	✓

- **Object Class**

Object class (in Java.lang) is the root of the Java class hierarchy. Every class in Java either directly or indirectly extends Object. It provides essential methods like toString(), equals(), hashCode(), clone() and several others that support object comparison, hashing, debugging, cloning and synchronization.

- Acts as the root of all Java classes
- Defines essential methods shared by all objects
- Supports thread communication (wait(), notify(), notifyAll())

Object ↑Every class directly or indirectly extends Object.

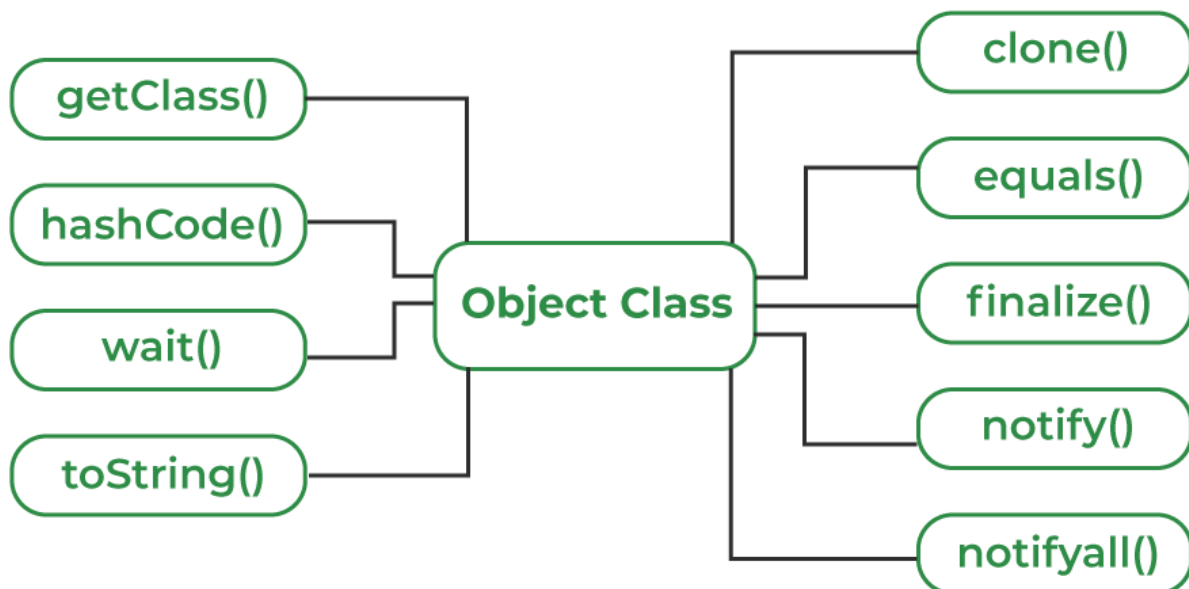
Also mention:

- equals()
- hashCode()
- toString()

are inherited by every class.

Object Class Methods

Object class provides multiple methods which are as follows:



- **this keyword**

In Java, this is a keyword that refers to the current object, the object whose method or constructor is being executed. It is mainly used to:

- Refer to the current class's instance variables and methods.
- Differentiate between instance variables and local variables when they have the same name.
- Pass the current object as an argument to methods or constructors.
- Call one constructor from another within the same class (using `this()`)
- Access instance variables and methods of the object on which the method or constructor is being invoked.

****Note:**** There is no concept of “transitivity” or “non-transitivity” — `this` always refers to the current object of a class

- **static keyword**

The static keyword in Java is used for memory management and belongs to the class rather than any specific instance. It allows members (variables, methods, blocks, and nested classes) to be shared among all objects of a class.

- Memory is allocated only once when the class is loaded.
- No object creation is needed to access static members; use the class name directly.
- Static methods and variables can't access non-static members directly.
- Static methods can't be overridden because they belong to the class, not instances.
Static methods are resolved at **compile time** (static binding), whereas overridden instance methods are resolved at **runtime** (dynamic binding).

Types of Static Members in Java

The static keyword can be applied to four main components:

1. Static Variables

A [static variable](#) is also known as a class variable. It is shared among all instances of the class and is used to store data that should be common for all objects.

2. Static Blocks

A [static block](#) is executed only once when the class is first loaded into memory. It is often used to initialize static variables or perform configuration tasks before the main method executes.

3. Static Methods

A [static method](#) belongs to the class rather than to any object. It can be called directly using the class name.

- Can access only static data directly.
- Cannot access instance variables or methods directly.
- Cannot use `this` or `super` keywords.

4. Static Nested Classes

A [static nested class](#) is a class declared as static inside another class. It can be accessed without creating an object of the outer class.

Note: The Inner class is accessed using the outer class name (Outer.Inner).

Static vs Non-Static

The table below demonstrates the difference between Static and Non-Static

Aspect	Static	Non-Static
Belongs To	Class (shared by all objects)	Individual object (unique per instance)
Memory Allocation	Created once when class is loaded	Created separately for each object
Access Method	Accessed using class name	Accessed using object reference
Can Access	Only other static members directly	Both static and non-static members
Common Use	Utility methods, constants, shared data	Object-specific data and behavior

Association / Aggregation / Composition

In object-oriented programming, relationships between classes play a crucial role in defining how objects interact with each other. Java, being an object-oriented language, provides mechanisms to model these relationships through association, aggregation, and composition.

- These relationships help model interactions between objects in real-world applications.
- They define the level of dependency and ownership between classes.
- Understanding them improves software design, maintainability, and code reusability.

Association

↓

USES-A

Teacher uses Projector

Aggregation

↓

HAS-A

Library has Books

Books can exist alone

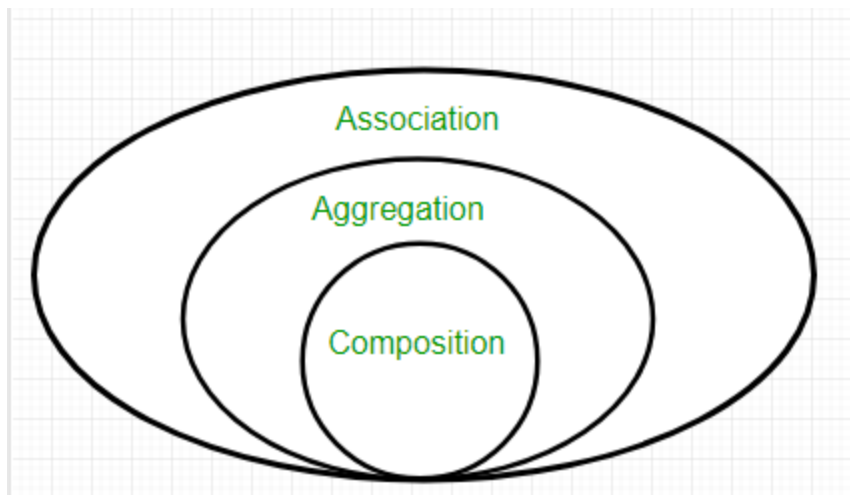
Composition

↓

PART-OF

Car has Engine

Engine dies with Car



Association

Association is a relationship between two independent classes where one object uses or interacts with another object. Both objects can exist independently of each other.

- Represents a Uses-A relationship.
- Objects are loosely coupled.
- Both classes can exist independently.

Types of Association

- ****Unidirectional Association:**** One class knows about another, but not vice versa. ****Example:**** A Student has a LibraryCard, but the LibraryCard doesn't know about

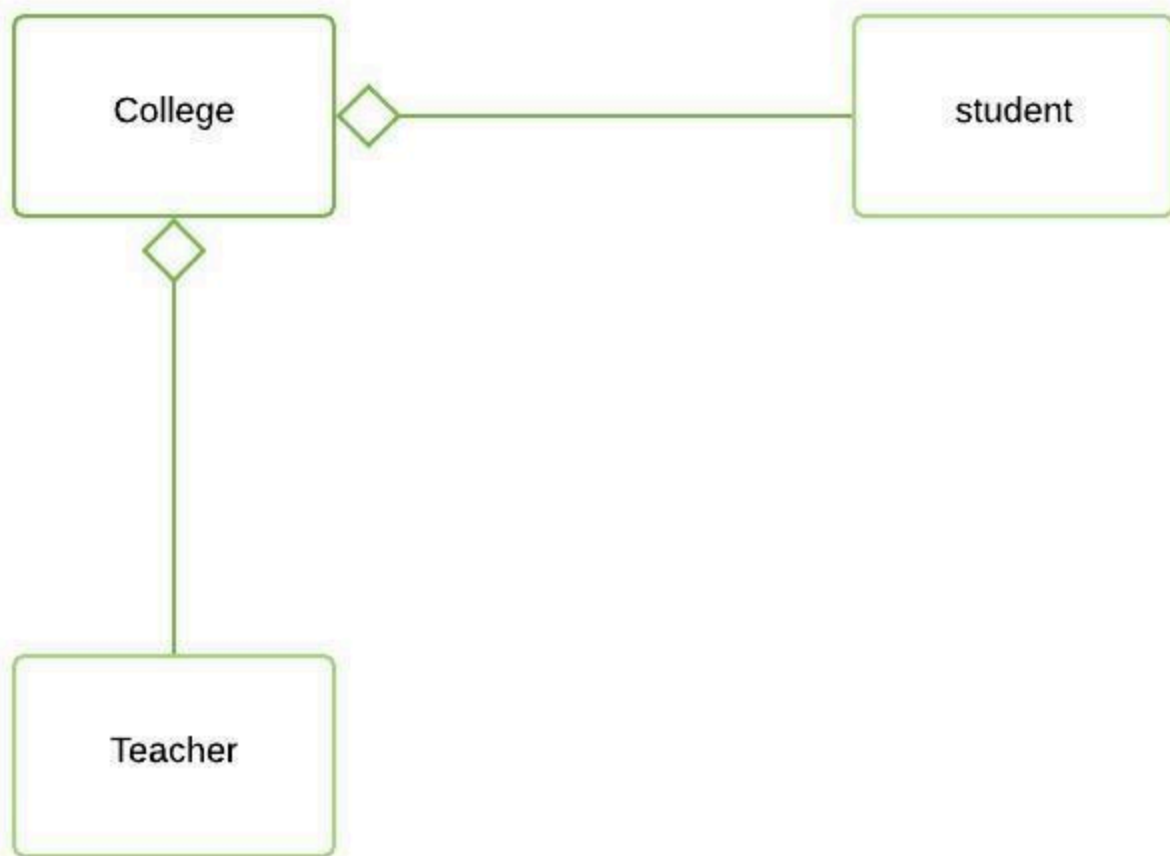
the Student.

- ****Bidirectional Association:**** Both classes know about and interact with each other. ****Example:**** A Teacher is assigned to a Classroom, and the Classroom knows its Teacher.

Aggregation

Aggregation is a special form of Association that represents a Has-A relationship with weak ownership. The contained object can exist independently of the container object.

- Represents a Has-A relationship.
- It is a weak association.
- Child objects can exist independently of the parent object.

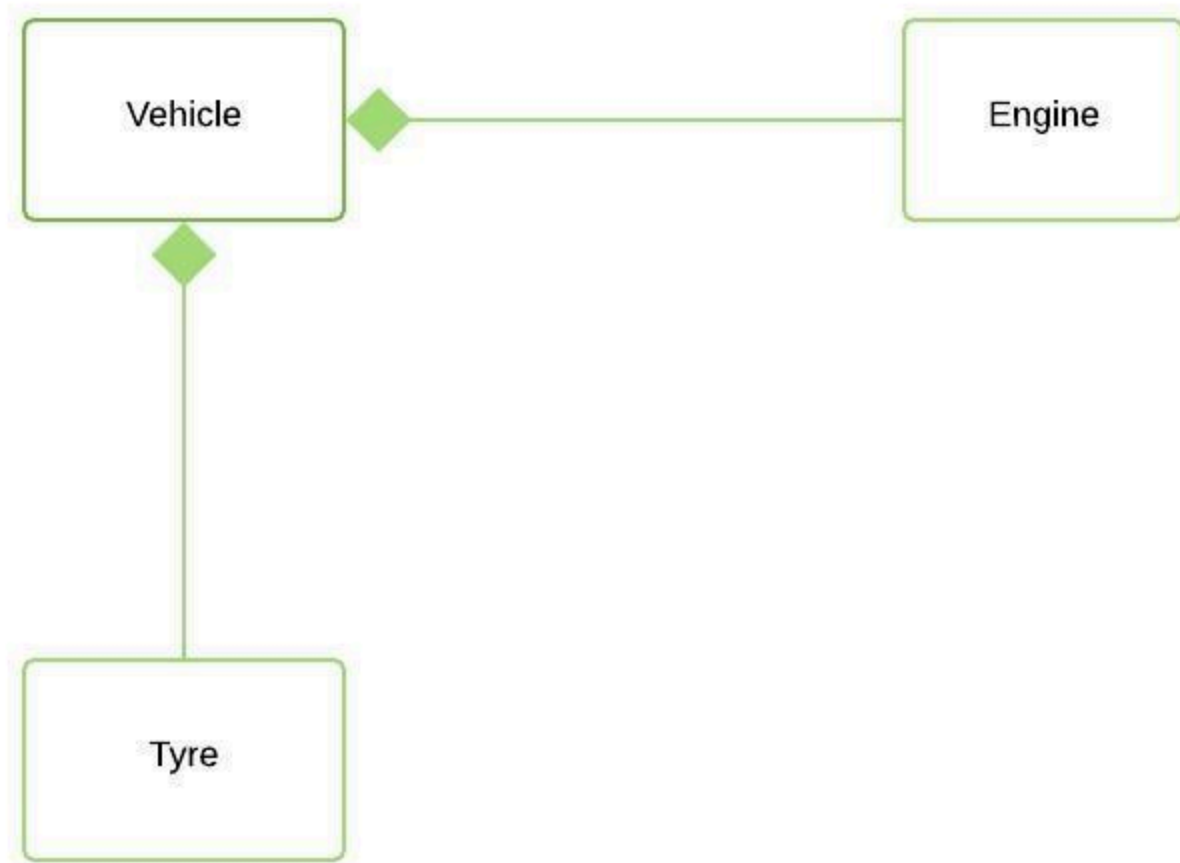


****Note**:** Code reuse is best achieved by aggregation.

Composition

Composition is a strong form of Aggregation that represents a Part-Of relationship. In Composition, the lifecycle of the contained object depends completely on the container object.

- Represents a Part-Of relationship.
- Strong ownership exists between objects.
- Child objects cannot exist without the parent object.



Difference Between Association, Aggregation and Composition

Feature	Association	Aggregation	Composition
Definition	General relationship between two classes	A special form of association with a "has-a" relationship	A stronger form of association with a "part-of" relationship
Dependency	Classes are related but can exist independently	Contained objects can exist independently of the container object	Contained objects cannot exist without the container object
Lifecycle	Independent lifecycles	Independent lifecycles	Dependent lifecycles
Ownership	No ownership implied	Shared ownership	Exclusive ownership
Strength	Weak	Moderate	Strong

Feature	Association	Aggregation	Composition
Cardinality	One-to-one, one-to-many, many-to-one, many-to-many	One-to-one, one-to-many, many-to-one, many-to-many	One-to-one, one-to-many
Example	Teacher and Student	Library and Books	Car and Engine
Representation	Uses a direct reference	Uses a reference to the contained object(s)	Contains instances of the contained object(s)
Illustrative Example Code	<pre>java class Teacher { private Student student; }</pre>	<pre>java class Library { private List books; }</pre>	<pre>java class Car { private Engine engine; }</pre>